

Detección de anomalías en mamografías mediante redes neuronales

Tomás Ávila, Álvaro García, Camilo Bonilla

16 de julio de 2026

1. Objetivos

La idea general es construir, por etapas, un sistema capaz de analizar mamografías. No pretendemos resolverlo todo de una vez: el plan es ir de lo general a lo específico en cuatro pasos. Primero, decidir si una mamografía es normal o presenta alguna anomalía. Segundo, cuando la hay, identificar de qué tipo es (calcificación, masa circunscrita, masa espiculada, distorsión arquitectural o asimetría). Tercero, ubicarla dentro de la imagen. Y cuarto, estimar su tamaño.

El orden no es caprichoso. Si el modelo no logra responder con confianza la pregunta más básica (¿hay algo raro acá o no?), las etapas siguientes, que dan por hecho que existe una anomalía, quedan sin piso. Por eso el grueso de este informe se concentra en la primera etapa: qué hicimos, con qué nos tropezamos y qué decisiones tomamos. En la sección 7 presentamos también una primera implementación de las etapas 2 y 3, que sirve para poner a prueba el diagnóstico general.

2. Datos

Trabajamos con el conjunto de la Mammographic Image Analysis Society (MIAS): 322 imágenes en formato PGM, en escala de grises, con una resolución nativa de unos 4320×2600 píxeles (algunas de 4320×2048). MIAS trae anotaciones bastante completas para cada imagen: tipo de tejido de fondo, clase de anomalía, severidad (benigna o maligna) y las coordenadas y el radio de la lesión. Para la primera etapa solo usamos la clase de anomalía, que convertimos en una etiqueta binaria: `normal` para las marcadas como `NORM` y `anomalía` para todo lo demás.

Al contar, quedaron 207 imágenes normales (64 %) contra 115 con anomalía (36 %). El conjunto está desbalanceado, y ese detalle, que al principio parecía menor, terminó marcando buena parte de los experimentos.

Otro dato que conviene tener presente desde ya: los radios de las lesiones van de 31 a 790 píxeles en la imagen original, con una mediana de 177. Las más pequeñas son las protagonistas de la discusión sobre resolución de la sección 5.3.

Los datos se repartieron en 70 % para entrenamiento, 15 % para validación y 15 % para test, manteniendo en cada subconjunto la misma proporción de normales y anomalías (Tabla 1).

Cuadro 1: Partición de los datos.

Subconjunto	Normal	Anomalía	Total
Entrenamiento	145	80	225
Validación	31	17	48
Test	31	18	49

3. Cómo lo abordamos

Optamos por Transfer Learning sobre una ResNet-50 preentrenada en ImageNet, cambiando la capa final por una cabeza de clasificación de dos clases. Con tan pocas imágenes, entrenar una red de ese tamaño desde cero no era una opción realista.

El preprocesamiento fue el habitual para este tipo de modelos: convertir la imagen a tres canales (los PGM traen uno solo), redimensionar, recortar el centro y normalizar con las estadísticas de ImageNet. Durante el entrenamiento agregamos aumento de datos: recortes aleatorios, rotaciones y volteo horizontal. Entrenamos con Adam y NLLLoss, inicialmente durante 10 épocas.

Todo esto arrancó en un computador con una GPU NVIDIA RTX 2050 de 4 GB de VRAM. Como se verá en la sección 5, esa memoria se quedó corta y el trabajo posterior se pasó a Google Colab, donde la GPU T4 ofrece 16 GB.

4. Los cuatro experimentos

Hicimos cuatro experimentos sobre el problema binario, cada uno motivado por lo que salió mal en el anterior.

4.1. Experimento 1: backbone congelado

Arrancamos por lo más sencillo: congelar toda la ResNet y entrenar únicamente la capa final. La precisión de validación se clavó en 64.58 % época tras época. Ese número no era casualidad: es exactamente la proporción de imágenes normales en validación (31 de 48). El modelo había aprendido el atajo más barato posible: responder “normal” siempre. La *accuracy* se veía decente en el papel, pero el recall de anomalías era cero. Fue la primera lección del proyecto: en un problema desbalanceado, la *accuracy* sola puede contar una historia completamente falsa.

4.2. Experimento 2: pesos por clase

Para quitarle el atajo, ponderamos la función de pérdida con pesos inversos a la frecuencia de cada clase, de modo que equivocarse con una anomalía saliera más caro. La precisión dejó de estancarse, sí, pero pasó a rebotar entre 35.42 % y 64.58 % (justo las dos proporciones de clase), y la pérdida se quedó pegada alrededor de 0.69 todas las épocas. Ese $0.69 \approx \ln 2$ es la pérdida de un clasificador binario que responde al azar. Los pesos eliminaron el sesgo, pero destaparon el problema de fondo: el modelo no estaba aprendiendo a distinguir nada.

4.3. Experimento 3: *fine-tuning* del último bloque

Si las características congeladas de ImageNet no servían para separar tejido sano de lesiones, había que dejar que la red las adaptara. Descongelamos el bloque `layer4` con un learning rate reducido (10^{-4}).

En entrenamiento la cosa mejoró de forma notoria: la precisión subió de 55 % a 90.67 % y la pérdida bajó de 0.71 a 0.24. Pero en validación pasó lo contrario: la pérdida creció de 0.74 a 1.35 mientras la precisión se quedaba rondando el 50 %. Cuando el entrenamiento mejora y la validación empeora al mismo tiempo, el diagnóstico es casi de manual: sobreajuste. El modelo se aprendió de memoria las 225 imágenes de entrenamiento.

En test el resultado lo confirma (tablas 2 y 3). De 18 anomalías reales, el modelo encontró 3. En un contexto médico ese es el número que importa, y 0.17 de recall dejaba claro que todavía estábamos lejos.

4.4. Experimento 4: mayor resolución (512×512) en Colab

Con la evidencia de que a 224×224 la señal de las lesiones se pierde (sección 5.3), trasladamos el entrenamiento a Google Colab y subimos la resolución de entrada a 512×512. No fue una sola corrida sino

Cuadro 2: Matriz de confusión en test (Experimento 3, 224×224).

	Predicho: anomalía	Predicho: normal
Real: anomalía	3	15
Real: normal	12	19

Cuadro 3: Métricas por clase en test (Experimento 3, 224×224).

Clase	Precisión	Recall	F1	Soporte
Anomalía	0.20	0.17	0.18	18
Normal	0.56	0.61	0.58	31
Accuracy			0.45	49

una secuencia, agregando piezas a medida que los resultados lo pedían: primero más épocas, luego *early stopping* y regularización (*weight decay* y aumento de datos reforzado). La Tabla 4 resume la evolución.

Cuadro 4: Evolución de las corridas del problema binario en test.

Corrida	Configuración	Recall anom.	Precisión	F1 anom.	Accuracy
Exp. 3 (local)	224, 10 ép.	0.17 (3/18)	0.20	0.18	0.45
512 v1	512, 10 ép.	0.00 (0/18)	0.00	0.00	0.63
512 v2	512, 20 ép.	0.22 (4/18)	0.40	0.29	0.59
512 v3	512, 20 ép. + early stop + reg.	0.28 (5/18)	0.38	0.32	0.57

La secuencia dejó tres hallazgos. El primero fue contraintuitivo: la primera corrida a 512 con solo 10 épocas dio recall cero, peor que a 224. La explicación es que a mayor resolución la optimización se vuelve más pesada y el modelo *subajusta*: con tan pocas épocas no alcanza a aprender ni el conjunto de entrenamiento. Al duplicar las épocas empezó a aprender de verdad. El segundo hallazgo es que, apenas el modelo aprende, el sobreajuste reaparece (era de esperarse con 225 imágenes), y ahí es donde el *early stopping* y la regularización pagan: la corrida v3 logró el mejor resultado de todo el proyecto, con recall de 0.28 y F1 de 0.32. La curva de pérdida de esa corrida (Figura 2) muestra el fenómeno con claridad, y la Figura 3 presenta la matriz de confusión del modelo resultante. Además, la precisión de anomalía de 0.38 indica que el modelo ya discrimina en lugar de adivinar.

El tercer hallazgo es el más importante. Los saltos de recall (0.17 → 0.22 → 0.28) se van achicando con cada mejora: la curva se está aplanando, como se ve en la Figura 1. Nuestra lectura es que alcanzamos el techo que impone el tamaño del dataset. Con 225 imágenes de entrenamiento, ningún ajuste de hiperparámetros va a producir saltos grandes; romper ese techo exige más datos.

5. Problemas principales

Los experimentos destaparon tres obstáculos. Los presentamos en orden, del más superficial al que consideramos el problema de fondo.

5.1. El desbalance de clases

Con 64 % de imágenes normales, predecir siempre “normal” regala un 64 % de *accuracy* sin aprender nada, que fue justo lo que pasó en el Experimento 1. La corrección aplicada fue doble: ponderar la pérdida con pesos inversos a la frecuencia y, sobre todo, cambiar de métrica. Dejamos de mirar la *accuracy* y pasamos a la matriz de confusión y al recall de la clase anomalía, que responde la pregunta que de verdad interesa: de las lesiones que existen, ¿cuántas estamos encontrando?

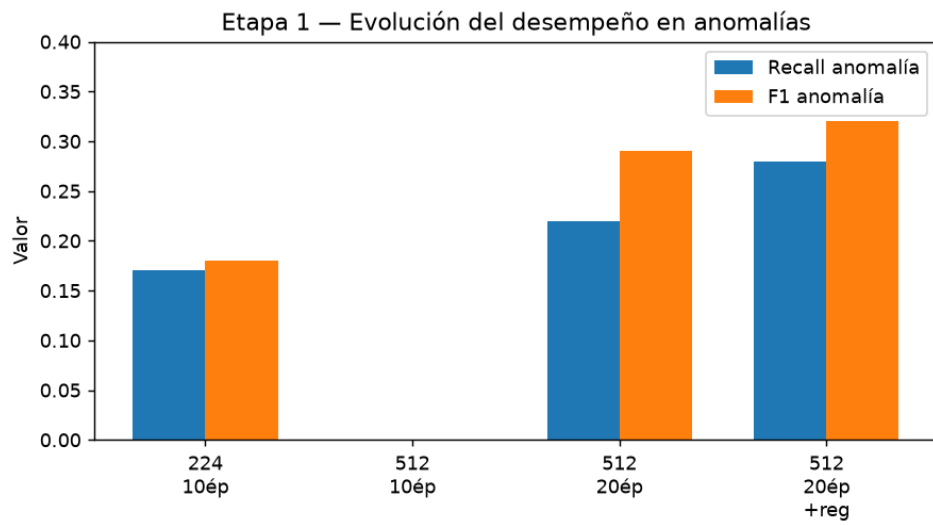


Figura 1: Evolución del recall y F1 de anomalía a lo largo de los cuatro experimentos. La mejora es incremental y decreciente, señal del techo por datos.

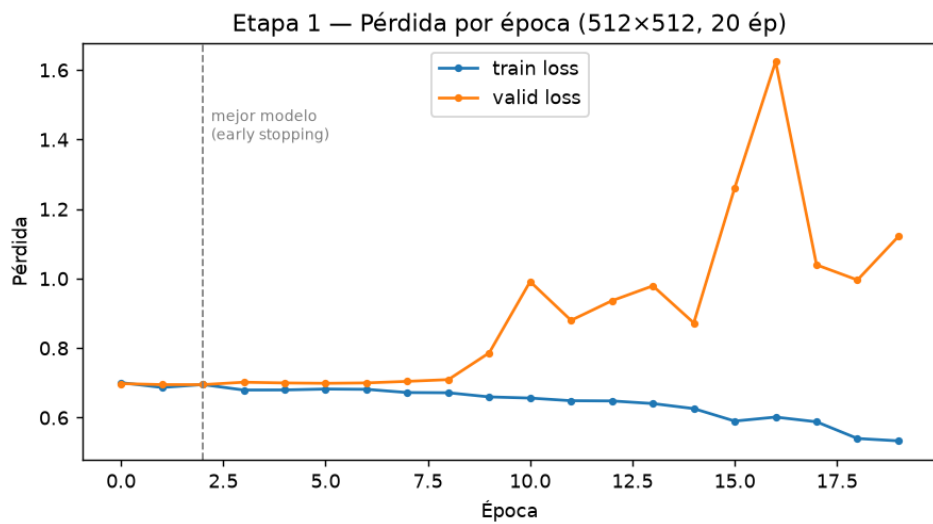


Figura 2: Pérdida de entrenamiento y validación (512×512, 20 épocas). La pérdida de validación se dispara mientras la de train baja: sobreajuste. El early stopping selecciona el modelo de la época 2.

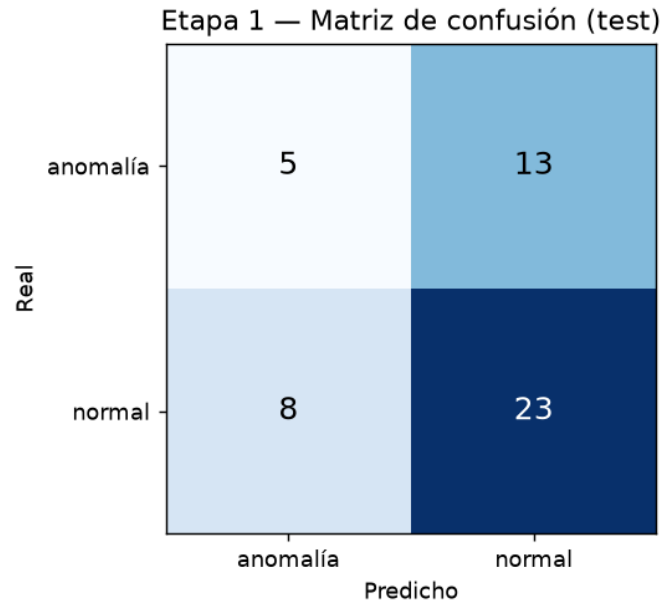


Figura 3: Matriz de confusión sobre test del mejor modelo (recall de anomalía 0.28: detecta 5 de 18 anomalías).

5.2. El sobreajuste

225 imágenes de entrenamiento son muy pocas para ajustar una ResNet-50 sin que termine memorizando, y el Experimento 3 lo dejó en evidencia. Las contramedidas (*early stopping*, que conserva el modelo de la mejor época de validación y no el de la última; *weight decay*; *dropout* en la cabeza; y aumento de datos reforzado) ya no son una promesa: el Experimento 4 midió su efecto y con ellas se obtuvo el mejor resultado. Queda pendiente probar un modelo más pequeño (ResNet-18), aunque no esperamos que cambie el panorama por sí solo.

5.3. La pérdida de resolución

Este fue el limitante que más condicionó la estrategia, hasta que el Experimento 4 reveló uno todavía más profundo.

Las mamografías nativas tienen unos 11.2 megapíxeles, pero el modelo las recibía a 224×224. Haciendo la cuenta, el modelo veía más o menos el 0.45 % de los píxeles originales; cada píxel de entrada resume un bloque de unos 10×10 píxeles reales. Para la lesión más pequeña del conjunto (radio de 31 px) eso significa quedar reducida a un puntico de unos 6 píxeles de diámetro, y las microcalcificaciones individuales, más pequeñas que su cluster, directamente desaparecen: quedan por debajo del tamaño de un píxel. Para completar, el recorte central bota cerca del 47 % del ancho de la mama, así que una lesión pegada al borde puede salir de la imagen antes de que el modelo la vea. Dicho sin rodeos: le estábamos pidiendo al modelo que detectara algo que ya no estaba en su entrada.

¿Y por qué no meter la imagen completa? Por la GPU. Medimos el consumo de VRAM en entrenamiento (ResNet-50, batch de 1, GPU de 4 GB); los resultados están en la Tabla 5.

La salida fue buscar un punto de equilibrio y mover el cómputo a Colab, cuya T4 cuadruplica la VRAM disponible. Elegimos 512×512 como resolución de trabajo. Con eso la lesión mediana pasa de unos 9 píxeles a unos 21, el batch se mantiene estable alrededor de 32 y el tiempo por época sigue siendo razonable. Consideramos resoluciones mayores (1024 o más), pero las descartamos por ahora: colapsan el batch, se alejan demasiado de la escala con la que se preentrenó ImageNet y aportan un nivel de detalle que la tarea binaria todavía no necesita. El Experimento 4 validó la decisión: subir a 512 mejoró el recall, pero también mostró que la resolución no era el único cuello de botella.

Cuadro 5: Consumo de VRAM en entrenamiento (ResNet-50, batch=1, GPU 4 GB).

Resolución de entrada	Megapíxeles	VRAM
224×224	0.1	0.49 GB
448×448	0.2	0.52 GB
896×896	0.8	1.54 GB
1344×1344	1.8	3.26 GB
2048×2048	4.2	sin memoria (OOM)
Nativa 4320×2600	11.2	sin memoria (OOM)

5.4. Una alternativa que dejamos en standby: parches

También evaluamos otra vía: en lugar de encoger la imagen completa, extraer parches a resolución nativa centrados en las lesiones, aprovechando las coordenadas que trae MIAS. La ventaja es evidente: se conserva el 100 % del detalle local con el mismo costo de memoria. El problema es metodológico: usar las coordenadas de la lesión para construir la entrada es, en la práctica, filtrar información de la etiqueta hacia el modelo. Evaluar eso honestamente exigiría un protocolo cuidadoso (por ejemplo, un barrido ciego sobre toda la imagen en test) para no inflar los resultados. Lo dejamos como línea futura; de hecho, para las etapas 3 y 4, donde localizar es parte del objetivo, este enfoque se vuelve el natural.

6. Limitaciones

225 imágenes de entrenamiento son muy pocas para el modelo que estamos usando, y el Experimento 4 mostró que ese es el techo real del desempeño. Además, incluso a 512×512 seguimos comprimiendo la imagen unas 8 veces respecto a la nativa, así que las lesiones más pequeñas siguen en riesgo de perderse. Y en el fondo hay una tensión conceptual: la detección de anomalías es un problema local, mientras que nosotros clasificamos la mamografía entera, lo que diluye la señal que importa en un mar de tejido normal.

7. Etapas 2 y 3: tipo de anomalía y localización

Sobre la misma base de transfer learning montamos versiones iniciales de las etapas 2 y 3, entrenadas únicamente con las imágenes que contienen anomalía (112 con anotación completa en MIAS). Conviene ser francos sobre su alcance: son demostraciones del pipeline, no modelos listos para usarse.

7.1. Etapa 2: clasificación del tipo de anomalía

La tarea es clasificar cada anomalía en uno de los seis tipos de MIAS: calcificación (CALC), masa circunscrita (CIRC), masa espiculada (SPIC), masa mal definida (MISC), distorsión arquitectural (ARCH) y asimetría (ASYM). El modelo es la misma ResNet-50 con `layer4` descongelado, ahora con una cabeza de seis salidas y pérdida *cross-entropy* ponderada por clase (los tipos también están desbalanceados, con entre 15 y 25 imágenes cada uno), más early stopping y regularización.

La limitación acá es estructural: quedan unas 80 imágenes de entrenamiento repartidas en seis clases, es decir, unas 13 por clase. Los resultados no dejan lugar a dudas: la accuracy en test fue de 0.17, que es exactamente lo que daría responder al azar entre seis clases ($1/6 \approx 0.167$). El entrenamiento mostró un sobreajuste extremo, con una pérdida de 0.011 en train contra 2.7 en validación; o sea, el modelo se memorizó el conjunto sin aprender nada transferible. La Figura 4 lo confirma visualmente: las predicciones se dispersan sin patrón. Esta etapa confirma el diagnóstico de la etapa 1 de forma todavía más marcada, y lo que demuestra es que el pipeline multiclase funciona de punta a punta, no que tengamos un clasificador confiable.

Etapa 2 — Tipo de anomalía (test, accuracy 0.17 $\approx$ azar)

Real \ Predicho	CALC	CIRC	SPIC	MISC	ARCH	ASYM
CALC	0	0	0	0	0	2
CIRC	2	0	0	1	0	1
SPIC	0	1	0	0	1	1
MISC	0	1	0	1	1	0
ARCH	1	0	1	0	1	0
ASYM	0	1	0	1	0	1

Figura 4: Matriz de confusión del clasificador de tipo (test). Las predicciones se dispersan sin patrón, coherente con un desempeño a nivel de azar.

7.2. Etapa 3: localización de la anomalía

Acá la tarea cambia de naturaleza: en lugar de clasificar, el modelo predice tres valores continuos (la posición (x, y) y el radio de la lesión) mediante regresión, con las coordenadas de MIAS como verdad de terreno. Usamos la misma ResNet-50 con una cabeza de tres salidas y pérdida *Smooth L1*, evaluando por error absoluto medio en píxeles.

Hay una salvedad importante que descubrimos en el proceso: el sistema de coordenadas del README de MIAS no calza exactamente con la resolución de nuestras imágenes (hay coordenadas que llegan a $y = 3542$ px cuando las imágenes miden unos 2600 px de alto). Mientras no se resuelva esa calibración, la etapa 3 debe leerse como una demostración del pipeline de regresión de posición, no como localización clínicamente validada.

Ambas etapas confirman, a mayor escala, el diagnóstico de la etapa 1: el factor limitante es la escasez de datos. Su valor está en demostrar que la arquitectura del sistema soporta las tareas del roadmap; lo que falta no es diseño sino material.

8. Lo que sigue

Para la etapa 1 quedan dos frentes: comparar el efecto de la resolución sobre el recall (512 contra 768) y probar ResNet-18 como modelo más liviano. Pero la prioridad clara, a la luz del techo encontrado, es ampliar los datos combinando MIAS con conjuntos más grandes como CBIS-DDSM, VinDr-Mammo o el de RSNA.

Para las etapas 2 y 3 el camino es el mismo (más datos) con dos tareas propias: calibrar correctamente el sistema de coordenadas de MIAS y, cuando la localización madure, retomar el enfoque por parches, que ahí encaja de forma natural. La etapa 4 (estimación del tamaño como regresión) se apoyará sobre la misma base.

9. Conclusión

En la etapa 1 el desempeño mejoró de forma incremental y medida a lo largo de cuatro experimentos: el recall de anomalías pasó de 0.17 a 0.28 (F1 de 0.18 a 0.32) tras subir la resolución a 512×512 y aplicar early stopping y regularización. Sigue siendo un número modesto, pero tenemos un diagnóstico con evidencia y cifras de por qué el enfoque estándar se queda corto. El desbalance de clases empuja al modelo a un atajo, los datos escasos lo llevan a memorizar, la reducción de resolución borra de la entrada lo que debería detectar y, en el fondo de todo, el techo lo pone el tamaño del conjunto: 322 imágenes no dan para más con las herramientas actuales. Las etapas 2 y 3, montadas como demostración del pipeline, confirman ese mismo diagnóstico a mayor escala. El sistema listo para escalar a datasets mayores junto con alguna alternativa (CBIS-DDSM, VinDr-Mammo, RSNA), que es la vía identificada para alcanzar un desempeño efectivo.

A. Reproducibilidad

El código está disponible en este repositorio: github.com/jbonillaro-lgtm/ProjectTL `main.py` para la versión local y el notebook de Colab para las corridas a 512×512. Las imágenes de MIAS no se redistribuyen; se obtienen de la fuente original.

La partición de los datos no depende de ninguna semilla: las carpetas `train/`, `valid/` y `test/` están fijas en disco y `ImageFolder` las lee tal cual, de modo que cualquiera que use esas mismas carpetas trabaja exactamente sobre la misma partición. Los entrenamientos, en cambio, no son reproducibles bit a bit, porque no se fijó semilla para la inicialización de la cabeza ni para el aumento de datos: los números pueden variar levemente entre corridas.

El entorno local fue PyTorch 2.13 con CUDA sobre Python 3.14 en un entorno virtual; en Colab se usó la GPU T4 de 16 GB con PyTorch preinstalado.

B. Referencias

Referencias

- [1] M. Alruwaili and W. Gouda, “Automated Breast Cancer Detection Models Based on Transfer Learning,” *Sensors*, vol. 22, no. 3, p. 876, 2022. doi: 10.3390/s22030876.